

Manual Técnico

Sistema de Comanda Digital v1.0

Control de Versiones

Versión	Fecha	Autor	Cambios
1.0	2026-05-24	Sistema	Versión inicial

Tabla de Contenidos

1. [Introducción](#)
 2. [Arquitectura del Sistema](#)
 3. [Requisitos del Sistema](#)
 4. [Instalación y Configuración](#)
 5. [Estructura de Directorios](#)
 6. [Base de Datos](#)
 7. [Front Controller y Enrutamiento](#)
 8. [Controladores](#)
 9. [Modelos](#)
 10. [Vistas](#)
 11. [Frontend — CSS](#)
 12. [Frontend — JavaScript](#)
 13. [API REST — Documentación](#)
 14. [Imágenes — Especificación Completa](#)
 15. [Pruebas \(PHPUnit\)](#)
 16. [Seguridad](#)
 17. [Mantenimiento](#)
 18. [Troubleshooting](#)
 19. [Referencia Rápida](#)
 20. [Bugs Conocidos](#)
-

1. Introducción

1.1 Descripción General

Sistema de Comanda Digital es una aplicación web para restaurantes que permite a los clientes seleccionar su mesa, elegir productos del menú (comidas y bebidas), realizar pedidos, y a los staff (cocina, meseros, caja) gestionar el flujo completo de atención.

El sistema está diseñado para una taquería llamada "**Taquería El Informático**" con roles de:

- **Admin:** Gestión completa (mesas, menú, usuarios, inventario, reportes)
- **Mesero:** Visualización y gestión de pedidos activos
- **Cocina:** Visualización de pedidos filtrados (solo tacos y postres)
- **Caja:** Procesamiento de pagos y cierre de pedidos
- **Cliente:** Selección de mesa y realización de pedidos (sin autenticación)

1.2 Tecnologías Utilizadas

Tecnología	Versión	Propósito
PHP	8.0+	Backend (sin framework)
MySQL / MariaDB	5.7+ / 10.4+	Base de datos
Apache	2.4+	Servidor web (mod_rewrite)
Docker	20.10+	Contenedorización
JavaScript	ES6	Frontend interactivo
CSS3	-	Estilos (vanilla, custom properties)
PHPUnit	10.5	Testing unitario

1.3 Patrones de Diseño

Patrón	Implementación	Ubicación
Front Controller	<code>index.php</code> recibe todas las peticiones	Raíz del proyecto
MVC	Controladores → Modelos → Vistas	<code>controllers/</code> , <code>models/</code> , <code>views/</code>
Singleton	<code>Database::getConnection()</code>	<code>config/database.php</code>
Observer	<code>NotificationManager</code> + <code>StockObserver</code>	<code>models/observers/</code>

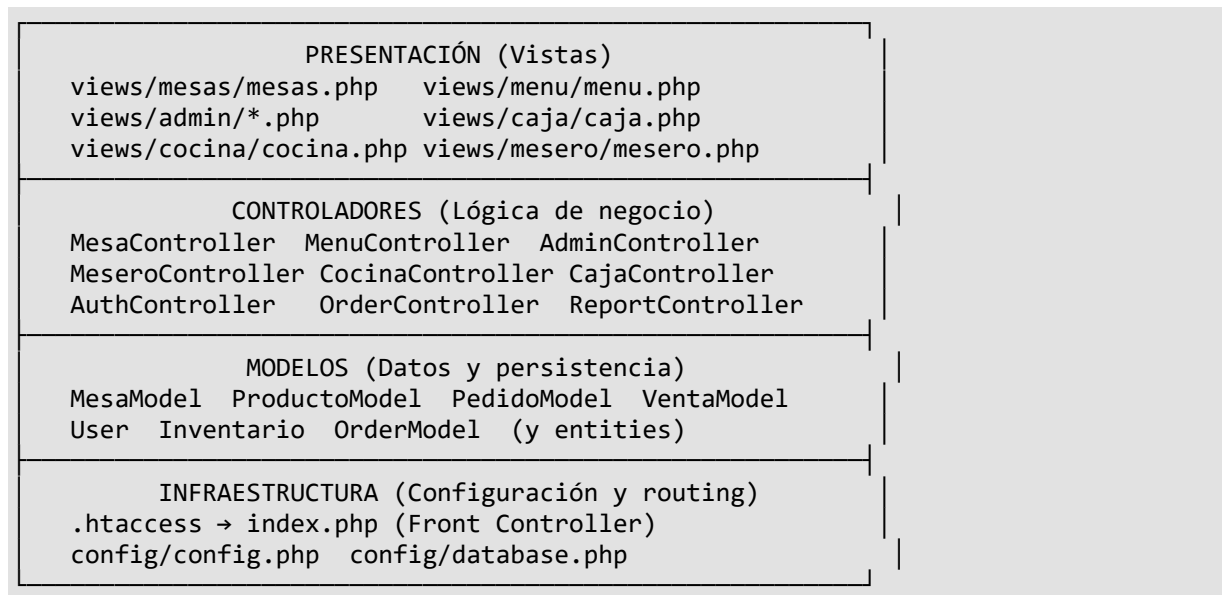
1.4 Glosario

Término	Definición
Comanda	Pedido realizado por un cliente
Mesa	Unidad física de atención al cliente
Pedido	Conjunto de productos solicitados por una mesa

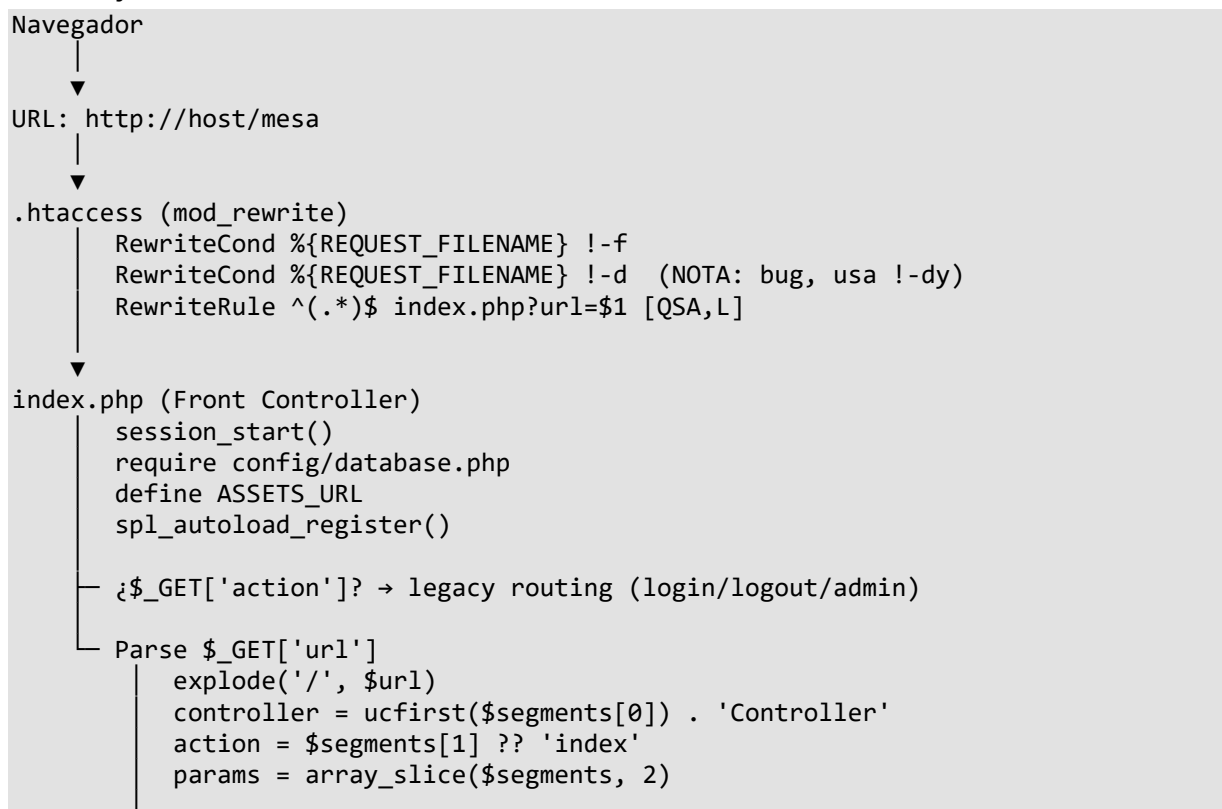
Detalle	Producto individual dentro de un pedido (con cantidad y estado)
Ticket	Resumen del pedido generado al confirmar
Venta	Registro de pago asociado a un pedido

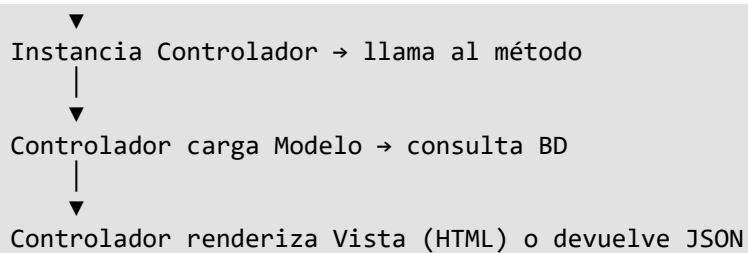
2. Arquitectura del Sistema

2.1 Diagrama de Capas



2.2 Flujo de Petición





2.3 Flujo de Pedido (Caso de uso completo)

1. Cliente visita /mesa
→ MesaController::index() → views/mesas/mesas.php
2. Cliente elige mesa libre
→ Redirección a /menu?mesa=N
→ MenuController::index() → views/menu/menu.php
3. Cliente agrega productos al carrito (JS)
→ carrito.js: cart[] array, renderCart(), showNotification()
4. Cliente finaliza pedido
→ carrito.js: enviarPedido() → POST /menu/confirmar
→ MenuController::confirmar()
→ PedidoModel::crearPedido() (transacción)
→ VentaModel::crearVentaPendiente()
→ Devuelve JSON con ticket
5. Cocina ve el pedido (polling cada 10s)
→ cocina.js: GET /cocina/obtenerPedidosActualizados
→ CocinaController::obtenerPedidosActualizados()
→ Filtra solo tacos y postres
6. Mesero ve el pedido (polling cada 5s)
→ mesero.js: GET /mesero/obtenerPedidosActualizados
7. Cocina cambia estado: pendiente → en_proceso → terminado
→ cocina.js: POST /cocina/actualizarDetalle
8. Mesero cambia estado: terminado → entregado
→ mesero.js: POST /mesero/actualizarDetalle
9. Caja procesa pago
→ POST /caja/pagar
→ CajaController::pagar()
→ VentaModel::pagarVenta()
→ PedidoModel::cerrarPedido()

2.4 Mapa de Enrutamiento Completo

URL	Método HTTP	Controlador	Acción	Parámetros	Tipo Respuesta
/o/mesa	GET	MesaController	index	-	HTML
/menu	GET	MenuController	index	?mesa=N	HTML

/menu/confirmar	POST	MenuController	confirmar	mesa_id, items (JSON)	JSON
/menu/solicitarAsistencia	POST	MenuController	solicitarAsistencia	mesa_id	JSON
/menu/getServerDateTime	GET	MenuController	getServerDateTime	-	JSON
/menu/enviarEmail	POST	MenuController	enviarEmail	pedido_id, email (JSON)	JSON
/mesero	GET	MeseroController	index	-	HTML
/mesero/obtenerPedidosActualizados	GET	MeseroController	obtenerPedidosActualizados	-	JSON
/mesero/actualizarDetalle	POST	MeseroController	actualizarDetalle	detalle_id, estado	JSON
/mesero/cerrarPedido	POST	MeseroController	cerrarPedido	pedido_id	JSON
/cocina	GET	CocinaController	index	-	HTML
/cocina/obtenerPedidosActualizados	GET	CocinaController	obtenerPedidosActualizados	-	JSON
/cocina/actualizarDetalle	POST	CocinaController	actualizarDetalle	detalle_id, estado	JSON
/caja	GET	CajaController	index	?mensaje=	HTML
/caja/pagar	POST	CajaController	pagar	venta_id, monto_pagado, metodo_pago	Redirect
/caja/cancelar	POST	CajaController	cancelar	venta_id	Redirect
/caja/borrarPendientes	POST	CajaController	borrarPendientes	-	Redirect
/order/save	POST	OrderController	save	mesa, items (JSON)	JSON
/order/getPendingOrders	GET	OrderController	getPendingOrders	-	JSON
/order/complete	POST	OrderController	complete	pedido_id (JSON)	JSON
/report/generarPDF	GET	ReportController	generarPDF	?tipo=diario/mensual/historial	HTML (download)

/login	GET/ POST	AuthContr oller	login	username, password	Redire ct
/logout	GET	AuthContr oller	logout	-	Redire ct
/admin	GET/ POST	AdminCon troller	index	?seccion=, ?accion=	HTML
?action=login	POST	AuthContr oller	login	username, password	Redire ct
?action=logout	GET	AuthContr oller	logout	-	Redire ct
?action=admin	GET/ POST	AdminCon troller	index	seccion, accion	HTML

3. Requisitos del Sistema

3.1 Entorno XAMPP

Componente	Versión Mínima	Verificación
PHP	8.0	php -v
Apache	2.4 (mod_rewrite)	`apachectl -M \
MySQL / MariaDB	5.7 / 10.4	mysql --version
Navegador	Chrome 90+, Firefox 88+, Edge 90+	-

Extensiones PHP requeridas:

- pdo y pdo_mysql (conexión a BD)
- mbstring (manejo de strings UTF-8)
- gd (manipulación de imágenes, si se requiere)

3.2 Entorno Docker

Componente	Versión Mínima
Docker Engine	20.10+
Docker Compose	2.0+

Puertos requeridos:

Puerto	Servicio	Uso
8081	Apache (web)	http://localhost:8081
3307	MySQL	Conexión externa opcional

4. Instalación y Configuración

4.1 Instalación en XAMPP (Linux)

```
# 1. Clonar/ubicar el proyecto
cp -r sistema_de_comanda_digital /opt/lampp/htdocs/proyecto/

# 2. Iniciar XAMPP
sudo /opt/lampp/lampp start

# 3. Importar base de datos
mysql -u root < /opt/lampp/htdocs/proyecto/sistema_de_comanda_digital/init-
db/sistema_comanda_digital_v1.sql

# 4. Verificar permisos de directorios
chmod -R 755
/opt/lampp/htdocs/proyecto/sistema_de_comanda_digital/public/images/platillos/
chmod -R 755 /opt/lampp/htdocs/proyecto/sistema_de_comanda_digital/logs/

# 5. Acceder en el navegador
# http://localhost/proyecto/sistema_de_comanda_digital/mesa
```

Configuración de **config/config.php** para XAMPP:

```
define('DB_HOST', 'localhost');      // Servidor MySQL local
define('DB_USER', 'root');           // Usuario MySQL
define('DB_PASS', '');               // Contraseña (vacía por defecto en XAMPP)
define('DB_NAME', 'sistema_comanda_digital_v1');
```

4.2 Instalación en Docker

```
# 1. Iniciar contenedores
docker-compose up -d

# 2. Verificar que MySQL esté listo (puede tomar 30s la primera vez)
docker-compose logs db

# 3. Verificar que la BD se importó
docker exec -it sistema_de_comanda_digital-db-1 mysql -uroot -proot \
-e "USE sistema_comanda_digital_v1; SHOW TABLES;"

# 4. Acceder en el navegador
# http://localhost:8081/mesa

# Comandos útiles
docker-compose down          # Detener
docker-compose logs -f       # Ver logs en tiempo real
docker exec -it sistema_de_comanda_digital-db-1 mysql -uroot -proot # Consola
MySQL

# Reconstruir desde cero (si hay cambios en Dockerfile)
docker-compose build --no-cache && docker-compose up -d

# Resetear la base de datos
docker-compose down -v && docker-compose up -d
```

4.3 Variables de Entorno

Variable	Docker (docker-compose.yml)	XAMPP (config.php)	Descripción
----------	-----------------------------	--------------------	-------------

DB_HOST	db	localhost	Host del servidor MySQL
DB_USER	root	root	Usuario MySQL
DB_PASS	root	' ' (vacío)	Contraseña MySQL
DB_NAME	sistema_comanda_digital_v1	sistema_comanda_digital_v1	Nombre de la BD

4.4 Credenciales por Defecto

Usuario	Contraseña	Rol
admin	password (o 123456)	Administrador
mesero1	password (o 123456)	Mesero
cocina1	password (o 123456)	Cocina
caja	password (o 123456)	Caja

⚠️ NOTA DE SEGURIDAD: Existe un backdoor con la contraseña **123456** que funciona para cualquier usuario. Ver sección [16. Seguridad](#).

5. Estructura de Directorios

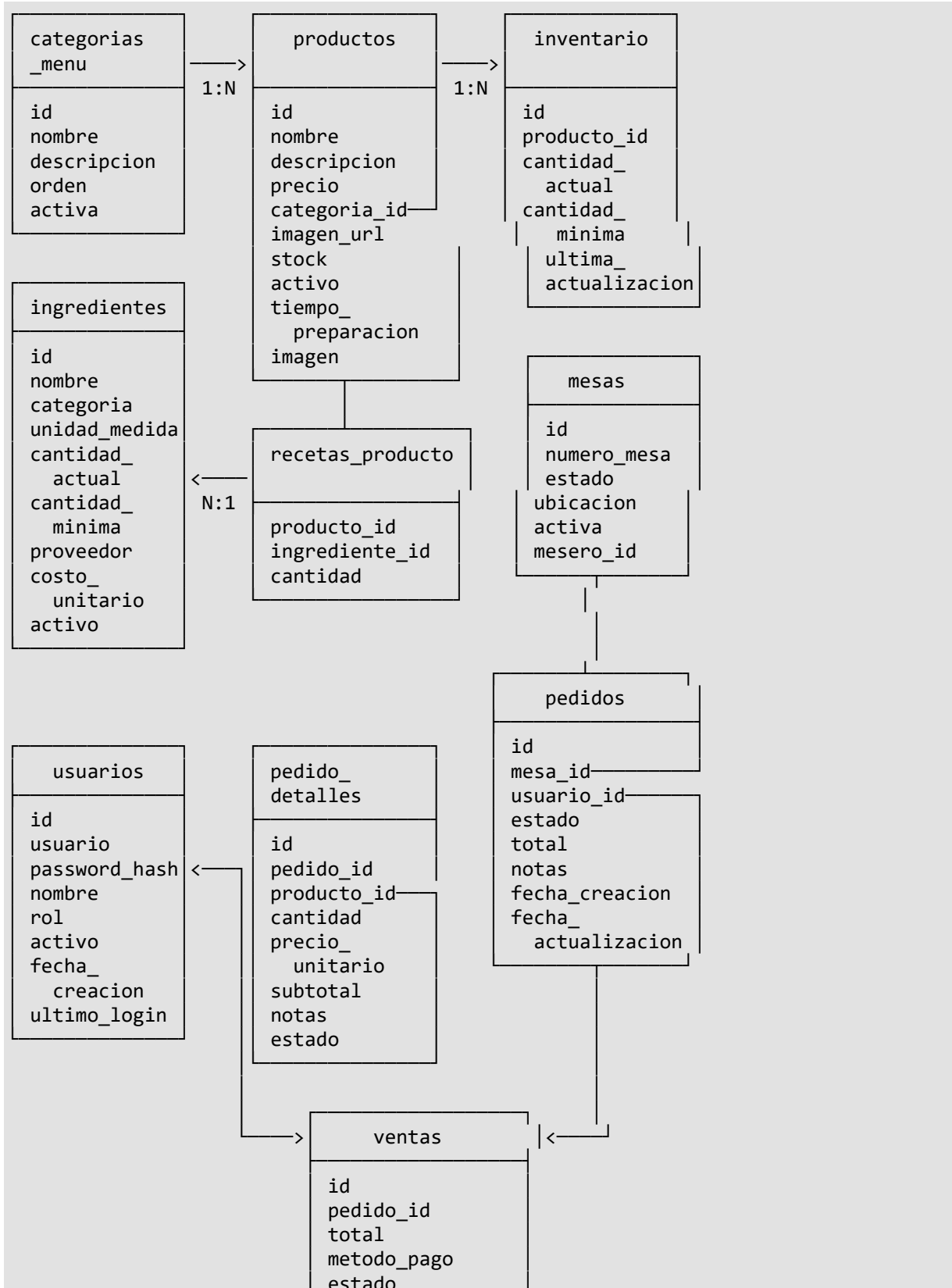
```
sistema_de_comanda_digital/
├── .htaccess                # Reescribe URLs a index.php?url=
├── index.php                # FRONT CONTROLLER (87 líneas)
├── composer.json            # PHPUnit 10.5, autoload PSR-4
├── composer.lock
├── package.json             # Playwright (solo dev)
├── Dockerfile               # php:8.2-apache + mod_rewrite + pdo_mysql
├── docker-compose.yml       # web (8081:80) + db (mysql:8.0, 3307:3306)
├── phpunit.xml              # Configuración de tests
├── generar_pdf.php          # Entry point para reportes PDF
├── config/
│   ├── config.php           # Constantes: BASE_URL, DB_HOST, DB_USER,
│   │                       DB_PASS, DB_NAME
│   ├── database.php         # Clase Database (Singleton PDO)
│   └── db.php               # Clase DatabasePDO (legacy, no singleton)
├── controllers/             # 9 controladores
│   ├── AuthController.php   # Login/logout
│   ├── AdminController.php  # CRUD completo (347 líneas)
│   ├── MenuController.php   # Menú público y pedidos (211 líneas)
│   ├── MesaController.php   # Listado de mesas (18 líneas)
│   ├── MeseroController.php # Panel mesero (54 líneas)
│   ├── CocinaController.php # Panel cocina (90 líneas)
│   ├── CajaController.php   # Caja registradora (99 líneas)
│   ├── OrderController.php  # API REST orders (88 líneas)
│   └── ReportController.php  # Generación de reportes (229 líneas)
```


models/	# 12 archivos de modelo
User.php	# Autenticación y CRUD usuarios
Mesa.php	# CRUD mesas (admin)
MesaModel.php	# Consultas mesas (público)
Producto.php	# CRUD productos + recetas
ProductoModel.php	# Consulta productos activos
Pedido.php	# Conteo pedidos activos
PedidoModel.php	# Ciclo de vida completo pedidos
OrderModel.php	# Modelo alternativo (legacy)
Venta.php	# Consulta ventas del día
VentaModel.php	# Ciclo de vida completo ventas
Inventario.php	# CRUD ingredientes
observers/	
NotificationManager.php	# Sujeto del patrón Observer
StockObserver.php	# Observador de stock bajo
views/	
auth/login.php	# Formulario de inicio de sesión
layout/	
header.php	# Header + sidebar del admin
sidebar.php	# Vacío (placeholder)
footer.php	# Footer admin + JS navegación
admin/	
dashboard.php	# KPIs (ventas hoy, pedidos activos, mesas)
mesas.php	# Gestión de mesas (alta/baja)
menu.php	# Gestión de productos + recetas
usuarios.php	# Gestión de usuarios
inventario.php	# Control de inventario
reportes.php	# Reportes y estadísticas
mesas/	
mesas.php	# Selección de mesa (público)
menu/	
menu.php	# Menú digital (público)
mesero/	
mesero.php	# Panel del mesero
cocina/	
cocina.php	# Panel de cocina
caja/	
caja.php	# Panel de caja
assets/	
css/	
estilos.css	# ESTILO PRINCIPAL (1445 líneas)
help-buttons.css	# Botones flotantes de ayuda
js/	
carrito.js	# Carrito de compras (375 líneas)
[ACTIVO]	
cocina.js	# Polling cocina [ACTIVO]
help-buttons.js	# Modal ayuda y asistencia [ACTIVO]
mesero.js	# Polling mesero [ACTIVO]
mesero-ayuda.js	# Notificaciones mesero [ACTIVO]
main.js	# Inicialización [NO UTILIZADO]
ModalManager.js	# Clase ES6 modales [NO UTILIZADO]
NotificationManager.js	# Clase ES6 notificaciones [NO UTILIZADO]
OrderManager.js	# Clase ES6 pedidos [NO UTILIZADO]
img/	# Imágenes de productos (7 archivos)
bistec.jpeg	
jamaica.avif	

- jamaica.png
- pastor.jpeg
- pastor.webp
- refrescos.jpeg
- suadero.jpeg
- public/
 - css/
 - admin.css # Panel admin (547 líneas)
 - caja.css # Panel caja (829 líneas)
 - cart.css # Estilos carrito (192 líneas)
 - cocina.css # Panel cocina (10 líneas)
 - global.css # Vacío (placeholder)
 - login.css # Login (139 líneas)
 - menu.css # Menú (33 líneas)
 - mesas.css # Mesas (226 líneas)
 - mesero.css # Mesero (21 líneas)
 - css/js/
 - admin.js # Logout y descarga PDF
 - images/platillos/ # IMÁGENES SUBIDAS
 - 1775218844_agua-de-jamaica.jpg
 - 1776984695_taco.jpg
 - 1776992752_tacouri.jpeg
- helpers/
 - Logger.php # Logging centralizado (archivos JSON)
- logs/ # Directorio de logs
 - app.log # Todos los niveles de log
 - error.log # Solo ERROR y CRITICAL
 - sistema.log # Eventos del Observer
- init-db/
 - sistema_comanda_digital_v1.sql # Schema completo + datos semilla (536 líneas)
- tests/
 - bootstrap.php # Configuración de tests
 - Unit/
 - ProductoTest.php # 6 tests
 - PedidoTest.php # 6 tests
 - MesaTest.php # 7 tests
 - LoggerTest.php # 4 tests
 - InventarioTest.php # 5 tests
- scripts/
 - backup.sh # Backup MySQL con compresión (retención 7 días)
 - run-tests.sh # Ejecutar PHPUnit
- helpers/Logger.php # Sistema de logging
- CHANGELOG.md # Historial de versiones
- GLOSARIO_TECNICO.md # Glosario de términos
- MANUAL_TECNICO.md # Este documento
- docsmd/ # Documentación adicional
 - PRD.md # Documento de Requisitos del Producto
 - PRODUCT_BACKLOG.md # Backlog del producto

6. Base de Datos

6.1 Diagrama Entidad-Relación



```
fecha_pago
fecha_creacion
usuario_id
```

6.2 Tabla: usuarios

Almacena los usuarios del sistema con sus roles.

Columna	Tipo	Nulo	Default	F K	Descripción
id	int(11)	NOT NUL L	AUTO_INCREMENT	-	Identificado r único
usuario	varchar(50)	NOT NUL L	-	-	Nombre de usuario (UNIQUE)
password_has h	varchar(255)	NOT NUL L	-	-	Hash bcrypt de la contraseña
nombre	varchar(100)	NOT NUL L	-	-	Nombre completo
rol	enum('admin','mesero','cocina','caja')	NOT NUL L	-	-	Rol del usuario
activo	tinyint(1)	NUL L	1	-	Soft-delete (1=activo, 0=inactivo)
fecha_creacion	timestamp	NUL L	current_timestamp ()	-	Fecha de creación
ultimo_login	timestamp	NUL L	NULL	-	Último inicio de sesión

Datos semilla:

i d	usuario	password_hash	nombre	rol
1	admin	\$2y\$10\$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2. uhewG/igi	Administrador Principal	admin
2	mesero 1	\$2y\$10\$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2. uhewG/igi	Juan Perez	mesero
3	cocina 1	\$2y\$10\$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2. uhewG/igi	Maria Garcia	cocina
4	caja	\$2y10\$d/5wQy2tJcgRxKBv7LEYq.8lt8QwMpGwZ533JM6VEY91M	Uriel	caja

		jbx.AVMa		
--	--	----------	--	--

Consultas típicas:

```
-- Autenticar usuario
SELECT * FROM usuarios WHERE usuario = :usuario AND activo = 1;

-- Obtener meseros activos
SELECT id, nombre FROM usuarios WHERE rol = 'mesero' AND activo = 1;

-- Crear usuario
INSERT INTO usuarios (usuario, password_hash, nombre, rol, activo)
VALUES (:usuario, :password_hash, :nombre, :rol, 1);
```

6.3 Tabla: **mesas**

Columna	Tipo	Nul o	Default	FK	Descripci ón
id	int(11)	NO T NU LL	AUTO_INCRE MENT	-	Identifica dor único
numero_m esa	varchar(10)	NO T NU LL	-	-	Número o código de mesa (UNIQUE)
estado	enum('libre','ocupada','reservada','mant enimiento')	NU LL	'libre'	-	Estado actual
ubicacion	varchar(100)	YES	NULL	-	Ubicación física (Terraza, Interior, etc.)
activa	tinyint(1)	NU LL	1	-	Soft- delete
mesero_id	int(11)	YES	NULL	→ usuarios .id	Mesero asignado

Datos semilla:

id	numero_mesa	estado	ubicacion	activa
1	M01	libre	Terraza	1
2	M02	libre	Interior	1
3	M03	libre	Sala Principal	1
4	M04	libre	Terraza	1
5	M05	libre	Sala VIP	1

6	M06	libre	Interior	0
10	M07	libre	Terraza	0

6.4 Tabla: **categorias_menu**

Columna	Tipo	Nulo	Default
id	int(11)	NOT NULL	AUTO_INCREMENT
nombre	varchar(100)	NOT NULL	-
descripcion	text	YES	NULL
orden	int(11)	NULL	0
activa	tinyint(1)	NULL	1

Datos semilla:

id	nombre	descripcion	orden
1	Tacos	Tipo de Taco	1
3	Bebidas	Refrescos y jugos	3
4	Postres	Deliciosos postres	4

6.5 Tabla: **productos**

Columna	Tipo	Nulo	Default	FK	Descripción
id	int(11)	NOT NULL	AUTO_INCREMENT	-	Identificador único
nombre	varchar(200)	NOT NULL	-	-	Nombre del producto
descripcion	text	YES	NULL	-	Descripción detallada
precio	decimal(10,2)	NOT NULL	-	-	Precio unitario
categoria_id	int(11)	YES	NULL	→ categorias_menu.id	Categoría del producto
imagen_url	varchar(500)	YES	NULL	-	URL externa de imagen (NO USADO)
stock	int(11)	NULL	0	-	Cantidad en inventario

activo	tinyint(1)	NUL L	1	-	Soft-delete
tiempo_preparacion	int(11)	NUL L	15	-	Tiempo estimado en minutos
imagen	varchar(255)	YES	NULL	-	Nombre del archivo de imagen local

Datos semilla:

id	nombre	descripcion	precio	categoria_id	stock	activo	tiempo	imagen
1	Taco campechano	taco campechano sin verduras ni queso	25.00	1	20	0	15	NULL
2	Taco de costilla	taco de costilla sin verduras	35.00	1	15	0	15	NULL
3	Taco de suadero con queso	Taco de suadero con queso sin verduras	28.00	1	12	0	15	NULL
4	Inca Kola	Refresco peruano 500ml	8.00	3	50	1	15	NULL
5	Chicha Morada	Bebida tradicional de maíz morado	7.00	3	30	0	15	NULL
6	Mazamorra Morada	Postre tradicional peruano	12.00	4	10	0	15	NULL
7	Arroz con Leche	Postre de arroz con canela	10.00	4	8	0	15	NULL
8	Taco al pastor	taco al pastor sencillo sin verdura ni queso	25.00	1	50	1	15	NULL

9	Taco al pastor con queso	taco al pastor con queso sin verduras	35.00	1	50	1	15	NULL
10	Taco de suadero	Taco de suadero sin queso ni verduras	35.00	1	50	0	15	NULL
11	taco de perro	orden de taco de perro	40.00	1	4	1	15	1776984695_taco.jpg
12	Taco Uriel	Taquito Uva	15.00	1	20	1	15	1776992752_tacouri.jpg

Consultas típicas:

```
-- Obtener productos activos con su categoría
SELECT p.*, c.nombre AS categoria
FROM productos p
JOIN categorias_menu c ON p.categoria_id = c.id
WHERE p.activo = 1;

-- Insertar nuevo producto
INSERT INTO productos (nombre, descripcion, precio, categoria_id, stock, activo,
tiempo_preparacion, imagen)
VALUES (:nombre, :descripcion, :precio, :categoria_id, :stock, 1, :tiempo,
:imagen);

-- Soft-delete producto
UPDATE productos SET activo = 0 WHERE id = :id;
```

6.6 Tabla: **ingredientes**

Columna	Tipo	Nul o	Default
id	int(11)	NO T NU LL	AUTO_INCREMENT
nombre	varchar(100)	NO T NU LL	-
categoria	enum('vegetales','carne','lacteos','granos','especias','bebidas','otros')	NU LL	'otros'
unidad_medida	enum('kg','gr','lt','ml','unidad','paquete')	NU LL	'kg'
cantidad_actual	decimal(10,3)	NU LL	0.000

cantidad_minima	decimal(10,3)	NULL	1.000
proveedor	varchar(100)	YES	NULL
costo_unitario	decimal(10,2)	NULL	0.00
activo	tinyint(1)	NULL	1
fecha_actualizacion	timestamp	NULL	ON UPDATE CURRENT_TIMESTAMP

Datos semilla: 10 ingredientes (Cebolla, Ajo, Tomate, Pollo, Carne de Res, Arroz, Papas, Limón, Aceite Vegetal, Sal).

6.7 Tabla: recetas_producto

Columna	Tipo	Nulo	Default	FK
id	int(11)	NOT NULL	AUTO_INCREMENT	-
producto_id	int(11)	NOT NULL	-	→ productos.id ON DELETE CASCADE
ingrediente_id	int(11)	NOT NULL	-	→ ingredientes.id ON DELETE CASCADE
cantidad	decimal(10,3)	NOT NULL	-	Cantidad del ingrediente

6.8 Tabla: pedidos

Columna	Tipo	Nulo	Default	FK
id	int(11)	NOT NULL	AUTO_INCREMENT	-
mesa_id	int(11)	NOT NULL	-	→ mesas.id
usuario_id	int(11)	NOT NULL	-	→ usuarios.id
estado	enum('pendiente','confirmado','en_preparacion','listo','entregado','cancelado')	NULL	'pendiente'	-

total	decimal(10,2)	NULL	0.00	-
notas	text	YES	NULL	-
fecha_creacion	timestamp	NULL	current_timestamp()	-
fecha_actualizacion	timestamp	NULL	ON UPDATE CURRENT_TIMESTAMP	-

6.9 Tabla: pedido_detalle

Columna	Tipo	Nulo	Default	FK
id	int(11)	NOT NULL	AUTO_INCREMENT	-
pedido_id	int(11)	NOT NULL	-	→ pedidos.id ON DELETE CASCADE
producto_id	int(11)	NOT NULL	-	→ productos.id ON DELETE CASCADE
cantidad	int(11)	NOT NULL	-	-
precio_unitario	decimal(10,2)	NOT NULL	-	Precio en el momento del pedido
subtotal	decimal(10,2)	NOT NULL	-	cantidad × precio_unitario
notas	text	YES	NULL	Notas especiales
estado	enum('pendiente','en_preparacion','listo','entregado')	NULL	'pendiente'	Estado individual del detalle

6.10 Tabla: **ventas**

Columna	Tipo	Nulo	Default	FK
id	int(11)	NOT NULL	AUTO_INCREMENT	-
pedido_id	int(11)	NOT NULL	-	→ pedidos.id
total	decimal(10,2)	NOT NULL	-	Total a pagar
metodo_pago	enum('efectivo','tarjeta','transferencia','mixto')	NOT NULL	-	Método de pago
estado	enum('pendiente','pagado','cancelado')	NULL	'pendiente'	Estado de la venta
fecha_pago	timestamp	YES	NULL	Cuándo se pagó
fecha_creacion	timestamp	NULL	current_timestamp()	Cuándo se creó
usuario_id	int(11)	YES	NULL	→ usuarios.id Quién procesó

6.11 Tabla: **alertas_sistema**

Columna	Tipo	Nulo	Default
id	int(11)	NOT NULL	AUTO_INCREMENT
tipo	varchar(50)	NOT NULL	-
mensaje	text	NOT NULL	-
nivel	enum('bajo','medio','alto')	NULL	'medio'
leida	tinyint(1)	NULL	0
fecha_creacion	timestamp	NULL	current_timestamp()

6.12 Tabla: **inventario** (legacy)

Columna	Tipo	Nulo	Default	FK
id	int(11)	NOT NULL	AUTO_INCREMENT	-
producto_id	int(11)	NOT	-	→ productos.id ON DELETE

		NULL		CASCADE
cantidad_actual	int(11)	NOT NULL	-	-
cantidad_minima	int(11)	NULL	5	-
ultima_actualizacion	timestamp	NULL	current_timestamp()	-

6.13 Resumen de Claves Foráneas

Origen	Destino	Comportamiento
productos.categoria_id	categorias_menu.id	Restrict
inventario.producto_id	productos.id	ON DELETE CASCADE
pedidos.mesa_id	mesas.id	Restrict
pedidos.usuario_id	usuarios.id	Restrict
pedido_detalle.pedido_id	pedidos.id	ON DELETE CASCADE
pedido_detalle.producto_id	productos.id	ON DELETE CASCADE
recetas_producto.producto_id	productos.id	ON DELETE CASCADE
recetas_producto.ingrediente_id	ingredientes.id	ON DELETE CASCADE
ventas.pedido_id	pedidos.id	Restrict
ventas.usuario_id	usuarios.id	Restrict
mesas.mesero_id	usuarios.id	Restrict

7. Front Controller y Enrutamiento

7.1 .htaccess

```

RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-dy # ⚠️ BUG: debería ser !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]

```

Funcionamiento:

- `RewriteCond %{REQUEST_FILENAME} !-f`: Si la URL NO corresponde a un archivo existente
- `RewriteCond %{REQUEST_FILENAME} !-d`: Si la URL NO corresponde a un directorio existente
- `RewriteRule ^(.*)$ index.php?url=$1`: Reescribe a index.php con el path como parámetro `url`

⚠️ **BUG CONOCIDO**: La condición `!-dy` es un typo. Debería ser `!-d`. Esto causa que la verificación de directorios nunca se active. Solo se verifica si el archivo existe, no si el directorio existe. Esto no afecta el funcionamiento normal del sistema porque no hay URLs que correspondan a directorios reales.

7.2 index.php — Front Controller (87 líneas)

Flujo de ejecución:

1. session_start()
 - └─ Inicia o reanuda la sesión PHP
2. require config/database.php
 - └─ Carga config.php (constantes DB, BASE_URL)
 - └─ Define clase Database (singleton PDO)
3. define('ASSETS_URL', BASE_URL . 'assets/')
 - └─ URL base para assets CSS/JS/imágenes
4. spl_autoload_register()
 - └─ Autoloader personalizado:
controllers/{class}.php
models/{class}.php
models/observers/{class}.php
5. Routing Legacy (\$_GET['action'])
 - └─ 'login' → AuthController::login()
 - └─ 'logout' → AuthController::logout()
 - └─ 'admin' → AdminController::index()
6. Routing Moderno (\$_GET['url'])
 - └─ Default: 'mesa'
 - └─ Sanitizado con FILTER_SANITIZE_URL
 - └─ Explode por '/'
 - └─ [0] = controller (ucfirst + 'Controller')
 - └─ [1] = action (default 'index')
 - └─ [2+] = params
 - └─ Rutas especiales:
 - └─ 'admin' → AdminController::index()
 - └─ 'login' → AuthController::login()
 - └─ 'logout' → AuthController::logout()
 - └─ Si controlador existe → call_user_func_array()
 - └─ Si no → redirect a /mesa

Cálculo de BASE_URL:

```
$protocol = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ? "https" :  
"http";  
$host = $_SERVER['HTTP_HOST'];  
$script_name = $_SERVER['SCRIPT_NAME'];  
$script_dir = dirname($script_name);  
  
// Normaliza:  
// XAMPP: http://localhost/proyecto/sistema_de_comanda_digital/  
// Docker: http://localhost:8081/
```

8. Controladores

8.1 AuthController (49 líneas)

Método	URL	Método HTTP	Parámetros	Respuesta
login()	?action=login	POST	username, password	Redirect según rol

logout()	?action=logout	GET	-	Redirect a login
----------	----------------	-----	---	------------------

Flujo de login:

```
function login() {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $userModel = new User();
        $usuario = $userModel->login($_POST['username'], $_POST['password']);

        if ($usuario) {
            session_regenerate_id(true); // Prevenir session fixation
            $_SESSION['usuario_id'] = $usuario['id'];
            $_SESSION['usuario_nombre'] = $usuario['nombre'];
            $_SESSION['usuario_rol'] = $usuario['rol'];

            // Redirect según rol
            switch ($_SESSION['usuario_rol']) {
                case 'admin': redirect a admin dashboard
                case 'mesero': redirect a /mesero
                case 'caja': redirect a /caja
                case 'cocina': redirect a /cocina
            }
        } else {
            $error = "Usuario o contraseña incorrectos";
        }
    }

    require 'views/auth/login.php';
}
```

8.2 AdminController (347 líneas)

Método	URL	Descripción
index()	/admin	Punto de entrada, rutea secciones
procesarFormulario()	(interno)	Procesa acciones POST
dashboard()	?seccion=dashboard	KPIs y tabla de productos
gestionMesas()	?seccion=mesas	Alta/baja de mesas
gestionMenu()	?seccion=menu	CRUD productos + recetas
gestionUsuarios()	?seccion=usuarios	CRUD usuarios
controlInventario()	?seccion=inventario	Gestión ingredientes
reportes()	?seccion=reportes	Reportes y estadísticas

Acciones del formulario (POST):

accion	Método manejador	Tabla afectada
agregar_mesa	agregarMesa()	mesas
eliminar_mesa	eliminarMesa()	mesas
agregar_producto	agregarProducto()	productos, recetas_producto
eliminar_producto	eliminarProducto()	productos

agregar_usuario	agregarUsuario()	usuarios
eliminar_usuario	eliminarUsuario()	usuarios
actualizar_inventario	actualizarInventario()	ingredientes
agregar_ingredient	agregarIngrediente()	ingredientes

Verificación de autenticación:

```
if (!isset($_SESSION['usuario_id']) || $_SESSION['usuario_rol'] !== 'admin') {
    header('Location: ' . BASE_URL . 'index.php?action=login');
    exit;
}
```

8.3 MenuController (211 líneas)

Método	URL/Endpoint	Método HTTP	Descripción
index()	/menu	GET	Muestra menú para una mesa específica
confirmar()	/menu/confirmar	POST	Crea pedido + venta pendiente + ticket
solicitarAsistencia()	/menu/solicitarAsistencia	POST	Registra solicitud de ayuda
getServerDateTime()	/menu/getServerDateTime	GET	Devuelve fecha/hora del servidor
enviarEmail()	/menu/enviarEmail	POST	Deshabilitado

Ejemplo de respuesta JSON de confirmar():

```
// Éxito (200):
{
    "success": true,
    "message": "Pedido enviado a caja. Ticket generado.",
    "pedido_id": 12,
    "total": 42.00,
    "ticket": "🍷 TAQUERÍA EL INFORMÁTICO\n===== \n\n Mesa: M01\n..."
}

// Error (400):
{
    "success": false,
    "message": "Error al crear el pedido: La mesa no existe"
}
```

8.4 MesaController (18 líneas)

Método	URL	Descripción
index()	/mesa	Lista todas las mesas activas

Renderiza `views/mesas/mesas.php` con datos de todas las mesas donde `activa = 1`.

8.5 MeseroController (54 líneas)

Método	URL/Endpoint	Método	Descripción
--------	--------------	--------	-------------

		HTTP	
index()	/mesero	GET	Panel del mesero
obtenerPedidosActualizados()	/mesero/obtenerPedidosActualizados	GET	JSON de pedidos activos
actualizarDetalle()	/mesero/actualizarDetalle	POST	Cambia estado de un detalle
cerrarPedido()	/mesero/cerrarPedido	POST	Marca pedido como entregado

8.6 CocinaController (90 líneas)

Método	URL/Endpoint	Método HTTP	Descripción
index()	/cocina	GET	Panel de cocina (filtrado)
obtenerPedidosActualizados()	/cocina/obtenerPedidosActualizados	GET	JSON de pedidos filtrados
actualizarDetalle()	/cocina/actualizarDetalle	POST	Cambia estado de un detalle

Filtro de cocina (`filtrarTacosYPostres()`): Solo muestra detalles cuyo nombre de producto contenga (case-insensitive):

- `taco`
- `postre`
- `flan`
- `helado`
- `pastel`

Máquina de estados de cocina:

pendiente → en_proceso → terminado (solo avance, no retroceso)

8.7 CajaController (99 líneas)

Método	URL/Endpoint	Método HTTP	Descripción
index()	/caja	GET	Panel de caja con resumen
pagar()	/caja/pagar	POST	Procesa pago y cierra pedido
cancelar()	/caja/cancelar	POST	Cancela venta pendiente
borrarPendientes()	/caja/borrarPendientes	POST	Elimina todas las ventas pendientes

Flujo de pago:

```
function pagar() {
    $venta_id = $_POST['venta_id'];
    $monto_pagado = floatval($_POST['monto_pagado']);
```



```

$metodo_pago = $_POST['metodo_pago'] ?? 'efectivo';

$venta = $ventaModel->obtenerVentaPorId($venta_id);

if ($monto_pagado >= $venta['total']) {
    $cambio = $monto_pagado - $venta['total'];
    $ventaModel->pagarVenta($venta_id, $metodo_pago, $monto_pagado,
$usuario_id);
    $pedidoModel->cerrarPedido($venta['pedido_id']);
    // Redirect con mensaje de éxito
} else {
    // Redirect con "Monto insuficiente"
}
}

```

⚠️ **NOTA:** `$usuario_id` está hardcodeado a `1` en `CajaController` (línea ~40).

8.8 OrderController (88 líneas) — API REST

Método	URL/Endpoint	Método HTTP	Body	Respuesta
save()	/order/save	POST	<code>{"mesa": 1, "items": [...]}</code>	<code>{"status": "success", "pedido_id": 12}</code>
getPendingOrders()	/order/getPendingOrders	GET	-	<code>{"status": "success", "orders": [...]}</code>
complete()	/order/complete	POST	<code>{"pedido_id": 12}</code>	<code>{"status": "success", "updated_rows": 1}</code>

⚠️ **NOTA:** `OrderModel::getPendingOrders()` está declarado pero **no implementado** — causará error fatal si se invoca.

8.9 ReportController (229 líneas)

Método	URL	Descripción
generarPDF(\$tipo_reporte)	/report/generarPDF?tipo=diario	Genera reporte HTML descargable

Tipos de reporte:

- `diario`: Ventas del día actual con desglose por producto
- `mensual`: Ventas agrupadas por día del mes actual
- `historial`: Últimos 100 pedidos

NOTA: A pesar del nombre "PDF", el reporte se genera como HTML descargable (no es un PDF real).

9. Modelos

9.1 User (83 líneas)

Método	Parámetros	Retorno	Descripción
--------	------------	---------	-------------

login(\$username, \$password)	string, string	array\	false
getAll()	-	array[]	Todos los usuarios activos
crear(\$data)	array (usuario, password, nombre_completo, rol)	int\	false
eliminar(\$id)	int	bool	Soft-delete
obtenerMeseros()	-	array[]	Usuarios con rol='mesero'

Método login():

```
function login($username, $password) {
    $sql = "SELECT * FROM usuarios WHERE usuario = :usuario AND activo = 1";
    $stmt = $this->conn->prepare($sql);
    $stmt->execute([':usuario' => $username]);
    $usuario = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($usuario && (password_verify($password, $usuario['password_hash']) ||
$password === '123456')) {
        return $usuario;
    }
    return false;
}
```

9.2 ProductoModel (21 líneas)

Método	Parámetros	Retorno	SQL
obtenerProductosActivos()	-	array[]	SELECT p.*, c.nombre AS categoria FROM productos p JOIN categorias_menu c ON p.categoria_id = c.id WHERE p.activo = 1

9.3 PedidoModel (114 líneas)

Método	Parámetros	Retorno	SQL
crearPedido(mesa_id, usuario_id, items)	int, int, array[]	int (pedido_id)	Transacción: INSERT pedidos + INSERT detalles
obtenerPedidosActivos()	-	array[]	SELECT * FROM pedidos WHERE estado NOT IN ('entregado', 'cancelado')
obtenerPedidosActivosConDetalles()	-	array[]	Pedidos + detalles con nombres de producto
actualizarEstadoDetalle(detalle_id, estado)	int, string	bool	UPDATE pedido_detalle SET estado = ? WHERE id = ?
obtenerPedidoPorId(pedido_id)	int	array\	false
cerrarPedido(pedido_id)	int	bool	UPDATE pedidos SET estado = 'entregado' WHERE id = ?

9.4 VentaModel (149 líneas)

Método	Parámetros	Retorno
crearVentaPendiente(pedido_id, total)	int, float	bool
pagarVenta(venta_id, metodo_pago, monto_pagado, usuario_id)	int, string, float, int	bool
obtenerVentasPendientes()	-	array[]
obtenerVentasPagadas()	-	array[]
obtenerDetallesPedido(pedido_id)	int	array[]
obtenerVentaPorId(venta_id)	int	array\
cancelarVenta(venta_id)	int	bool
borrarVentasPendientes()	-	bool
obtenerVentasCanceladas()	-	array[]
obtenerResumen()	-	array (total_ventas, pagadas_count, pagadas_total, pendientes_count, pendientes_total, canceladas_count, canceladas_total, ventas_total)

9.5 MesaModel (24 líneas)

Método	Parámetros	Retorno	SQL
obtenerMesasActivas()	-	array[]	<code>SELECT * FROM mesas WHERE activa = 1</code>
mesaExiste(mesa_id)	int	bool	<code>SELECT id FROM mesas WHERE id = ? AND activa = 1</code>

9.6 Observers

NotificationManager (Subject):

```
class NotificationManager {
    private $observers = [];

    public function attach($observer) { $this->observers[] = $observer; }
    public function detach($observer) { /* remueve del array */ }
    public function notify($event, $data) {
        foreach ($this->observers as $observer) {
            $observer->update($event, $data);
        }
    }
}
```

StockObserver (Observer):

Evento	Manejador	Acción
stock_bajo	manejarStockBajo()	Log + INSERT alertas_sistema
producto_agotado	manejarProductoAgotado()	Log

inventario_actualizado	registrarCambioInventario()	Log
mesa_ocupada	registrarOcupacionMesa()	Log
pedido_creado	registrarNuevoPedido()	Log

Los logs del observer se escriben en [logs/sistema.log](#).

10. Vistas

10.1 Vistas Públicas (sin autenticación)

Vista	URL	Archivo CSS	Archivo JS	Descripción
Selección de mesa	/mesa	mesas.css, estilos.css	-	Tarjetas de mesa con estado
Menú digital	/menu?mesa=N	estilos.css, help-buttons.css	carrito.js, help-buttons.js	Productos con imágenes, categorías, carrito
Login	?action=login	login.css	-	Formulario de autenticación

10.2 Vistas de Staff (con autenticación)

Vista	URL	Archivo CSS	Archivo JS	Descripción
Panel mesero	/mesero	mesero.css	mesero.js, mesero-ayuda.js	Pedidos activos con controles
Panel cocina	/cocina	cocina.css	cocina.js	Pedidos filtrados (tacos/postres)
Panel caja	/caja	caja.css	-	Ventas pendientes/pagadas/canceladas

10.3 Vistas de Administración

Vista	Sección	Archivo CSS	Archivo JS	Descripción
Dashboard	?seccion=dashboard	admin.css	admin.js	KPIs, productos
Mesas	?seccion=mesas	admin.css	admin.js	CRUD mesas
Menú	?seccion=menu	admin.css	admin.js	CRUD productos + recetas
Usuarios	?seccion=usuarios	admin.css	admin.js	CRUD usuarios
Inventario	?seccion=inventario	admin.css	admin.js	CRUD ingredientes
Reportes	?seccion=reportes	admin.css	admin.js	Reportes diarios/mensuales/historial

11. Frontend — CSS

11.1 Sistema de Diseño (CSS Custom Properties)

```
:root {
  --primary-color: #8B0000;      /* Rojo vino oscuro */
  --secondary-color: #DC143C;    /* Rojo carmesí */
  --text-color: #333;
  --light-text: #fff;
  --background-color: #f9f9f9;
  --card-bg: #fff;
  --border-color: #e0e0e0;
}
```

11.2 Catálogo de Componentes

11.2.1 Header

```
header {
  background: linear-gradient(135deg, var(--primary-color) 0%, var(--secondary-color) 100%);
  color: var(--light-text);
  padding: 1rem;
  position: sticky;
  top: 0;
  z-index: 100;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.2);
}
```

11.2.2 Tarjeta de Mesa (mesas.css)

```
.mesa-card {
  background: var(--card-bg);
  border-radius: 16px;
  padding: 24px 16px;
  text-align: center;
  cursor: pointer;
  transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
  animation: cardFadeIn 0.5s ease both;
  border: 2px solid transparent;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

.mesa-card.libre:hover {
  transform: translateY(-8px);
  box-shadow: 0 12px 30px rgba(76, 175, 80, 0.2);
  border-color: #4CAF50;
}
```

11.2.3 Tab de Categoría (estilos.css)

```
.category-tab {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 10px 20px;
  border: 2px solid var(--primary-color);
  border-radius: 25px;
  background: white;
  color: var(--primary-color);
}
```

```

    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
}

.category-tab.active {
    background: linear-gradient(135deg, var(--primary-color), var(--secondary-color));
    color: white;
    border-color: transparent;
    box-shadow: 0 4px 15px rgba(139, 0, 0, 0.3);
}

```

11.2.4 Card de Producto (estilos.css)

```

.menu-item {
    background-color: var(--card-bg);
    border-radius: 15px;
    padding: 1.5rem;
    box-shadow: 0 6px 15px rgba(0,0,0,0.1);
    transition: all 0.3s ease;
    text-align: center;
    border: 2px solid transparent;
    position: relative;
    overflow: hidden;
    animation: bounceIn 0.6s ease;
}

.menu-item:hover {
    transform: translateY(-10px);
    box-shadow: 0 12px 25px rgba(0,0,0,0.15);
    border-color: var(--primary-color);
}

```

11.2.5 Imagen de Producto (estilos.css)

```

.item-image {
    width: 100%;
    height: 200px;
    overflow: hidden;
    border-radius: 10px;
    margin-bottom: 1.2rem;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #f8f9fa;
    position: relative;
}

```

11.2.6 Carrito Flotante (estilos.css)

```

.floating-cart {
    position: fixed;
    bottom: 25px;
    right: 25px;
    background: linear-gradient(135deg, var(--primary-color) 0%, var(--secondary-color) 100%);
    color: white;
    border-radius: 50%;
    width: 70px;
}

```

```

height: 70px;
display: flex;
align-items: center;
justify-content: center;
box-shadow: 0 6px 20px rgba(0, 0, 0, 0.3);
cursor: pointer;
z-index: 1000;
transition: all 0.3s ease;
border: 3px solid white;
}

```

11.2.7 Modal (estilos.css)

```

.cart-modal, .modal {
  display: none;
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background: linear-gradient(135deg, rgba(0,0,0,0.6) 0%, rgba(139,0,0,0.4) 100%);
  z-index: 1001;
  justify-content: center;
  align-items: center;
  backdrop-filter: blur(8px);
  animation: fadeIn 0.3s ease-out;
}

```

11.3 Animaciones

```

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-20px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes slideIn {
  from { transform: scale(0.8) translateY(-20px); opacity: 0; }
  to { transform: scale(1) translateY(0); opacity: 1; }
}

@keyframes bounceIn {
  0% { transform: scale(0.3); opacity: 0; }
  50% { transform: scale(1.05); }
  70% { transform: scale(0.9); }
  100% { transform: scale(1); opacity: 1; }
}

@keyframes cardFadeIn {
  from { opacity: 0; transform: translateY(30px) scale(0.95); }
  to { opacity: 1; transform: translateY(0) scale(1); }
}

```

11.4 Archivos CSS

Archivo	Líneas	Propósito
<code>assets/css/estilos.css</code>	1445	Estilo principal (header, menú, carrito, modales, notificaciones)
<code>assets/css/help-buttons.css</code>	44	Botones flotantes de ayuda y asistencia

<code>public/css/admin.css</code>	547	Panel de administración (sidebar, dashboard, tablas)
<code>public/css/caja.css</code>	829	Panel de caja (tarjetas de venta, resumen, animaciones)
<code>public/css/cart.css</code>	192	Estilos del carrito
<code>public/css/cocina.css</code>	10	Panel de cocina (mínimo)
<code>public/css/login.css</code>	139	Página de login (fondo gradiente, glassmorphism)
<code>public/css/menu.css</code>	33	Menú (estilos básicos)
<code>public/css/mesas.css</code>	226	Selección de mesas (grid, tarjetas, animaciones)
<code>public/css/mesero.css</code>	21	Panel de mesero
<code>public/css/global.css</code>	0	Placeholder (vacío)

12. Frontend — JavaScript

12.1 carrito.js (375 líneas) — Principal componente interactivo

Estado global:

```
const cart = []; // Array de {id, nombre, precio, cantidad}
let currentPedidoId = null; // ID del último pedido creado
```

DOM References:

Variable	Elemento	Propósito
<code>cartItemsContainer</code>	<code>#cartItemsContainer</code>	Lista de items en el modal del carrito
<code>cartTotal</code>	<code>#cartTotal</code>	Total del carrito
<code>cartBadge</code>	<code>#cartBadge</code>	Badge flotante con conteo
<code>cartData</code>	<code>#cartData</code>	Hidden field con JSON del carrito
<code>floatingCart</code>	<code>#floatingCart</code>	Botón flotante del carrito
<code>cartModal</code>	<code>#cartModal</code>	Modal del carrito
<code>closeCart</code>	<code>#closeCart</code>	Botón cerrar carrito
<code>checkoutBtn</code>	<code>#checkoutBtn</code>	Botón finalizar pedido
<code>pedidoForm</code>	<code>#pedidoForm</code>	Formulario de envío
<code>orderModal</code>	<code>#orderModal</code>	Modal de confirmación
<code>orderDetails</code>	<code>#orderDetails</code>	Contenedor del ticket

Eventos:

```
// 1. Agregar producto al carrito
document.querySelectorAll('.add-to-cart').forEach(btn => {
  btn.addEventListener('click', () => {
    const id = btn.dataset.id;
    const nombre = btn.dataset.nombre;
    const precio = parseFloat(btn.dataset.precio);
```



```

        const existingItem = cart.find(item => item.id === id);
        if (existingItem) {
            existingItem.cantidad++;
        } else {
            cart.push({ id, nombre, precio, cantidad: 1 });
        }
        renderCart();
        showNotification(`${nombre} agregado al carrito`);
    });
});

// 2. Abrir carrito (botón flotante)
floatingCart.addEventListener('click', () => cartModal.style.display = 'flex');

// 3. Cerrar carrito
closeCart.addEventListener('click', () => cartModal.style.display = 'none');

// 4. Finalizar pedido → enviarPedido()
checkoutBtn.addEventListener('click', () => {
    if (cart.length === 0) {
        showNotification('El carrito está vacío');
        return;
    }
    enviarPedido();
});

```

Flujo de enviarPedido():

```

function enviarPedido() {
    cartData.value = JSON.stringify(cart);

    fetch(pedidoForm.action, {
        method: 'POST',
        body: new FormData(pedidoForm)
    })
    .then(response => response.json().then(data => ({status: response.status,
body: data})))
    .then(result => {
        if (result.status >= 200 && result.status < 300 && result.body.success) {
            currentPedidoId = result.body.pedido_id;
            showOrderConfirmation(result.body.ticket, result.body.total);
        } else {
            showNotification(result.body.message || 'Error al enviar el pedido');
        }
    })
    .catch(error => {
        showNotification('Error de conexión');
        showOrderConfirmation(); // Fallback: muestra ticket con datos locales
    });
}

```

Renderizado del carrito (renderCart):

```

function renderCart() {
    cartItemsContainer.innerHTML = '';
    let total = 0;

    if (cart.length === 0) {
        cartItemsContainer.innerHTML = '<div class="empty-cart-message">Tu carrito

```

```

está vacío</div>';
    cartTotal.textContent = '$0.00';
    cartBadge.textContent = '0';
    return;
}

cart.forEach(item => {
    const itemTotal = item.precio * item.cantidad;
    total += itemTotal;

    const div = document.createElement('div');
    div.className = 'cart-item';
    div.innerHTML = `
        <div class="cart-item-info">
            <div class="cart-item-name">${item.nombre}</div>
            <div class="cart-item-price">${item.precio.toFixed(2)}</div>
            <div class="cart-item-controls">
                <button class="quantity-btn decrease-btn" data-
id="${item.id}">-</button>
                <span class="quantity-display">${item.cantidad}</span>
                <button class="quantity-btn increase-btn" data-
id="${item.id}">+</button>
                <button class="cart-item-remove" data-id="${item.id}"><i
class="fas fa-trash"></i></button>
            </div>
        </div>
        <div class="cart-item-total">${itemTotal.toFixed(2)}</div>
    `;
    cartItemsContainer.appendChild(div);
});

cartTotal.textContent = ` ${total.toFixed(2)} `;
cartBadge.textContent = cart.reduce((sum, item) => sum + item.cantidad, 0);
cartData.value = JSON.stringify(cart);
addCartItemEventListeners();
}

```

12.2 cocina.js (172 líneas) — Polling de cocina

```

const ESTADOS_MAP = {
    'pendiente': 'en_proceso',
    'en_proceso': 'terminado',
    'terminado': null // Estado final
};

function actualizarPedidos() {
    fetch(COCINA_URL) // COCINA_URL = /cocina/actualizarDetalle (asumido)
        .then(r => r.json())
        .then(data => { if (data.success) actualizarVistaPedidos(data.pedidos);
    });
}

function cambiarEstado(element) {
    const detalleId = element.dataset.detalleId;
    const estadoActual = element.dataset.currentState;
    const siguienteEstado = ESTADOS_MAP[estadoActual];

    if (!siguienteEstado) return;

    const formData = new FormData();

```

```

    formData.append('detalle_id', detalleId);
    formData.append('estado', siguienteEstado);

    fetch(ESTADO_URL, { method: 'POST', body: formData })
      .then(r => r.json())
      .then(data => { if (data.success) actualizarPedidos(); });
  }

  // Inicialización
  setInterval(actualizarPedidos, 10000);
  document.addEventListener('DOMContentLoaded', actualizarPedidos);

```

12.3 mesero.js (132 líneas) — Polling de mesero

```

const MESERO_URL = BASE_URL + 'index.php?url=mesero/';

function actualizarPedidos() {
  fetch(MESERO_URL + 'obtenerPedidosActualizados')
    .then(r => r.json())
    .then(data => { if (data.success) actualizarVistaPedidos(data.pedidos);
  });
}

function cerrarPedido(idPedido) {
  const formData = new FormData();
  formData.append('pedido_id', idPedido);
  fetch(MESERO_URL + 'cerrarPedido', { method: 'POST', body: formData })
    .then(r => r.json())
    .then(data => { if (data.success) actualizarPedidos(); });
}

// Inicialización
setInterval(actualizarPedidos, 5000);
window.addEventListener('focus', actualizarPedidos);

```

12.4 Archivos JavaScript NO Utilizados (dead code)

Los siguientes archivos están en el repositorio pero **no se cargan en ninguna vista**:

Archivo	Líneas	Propósito
assets/js/main.js	26	Inicialización (referencia clases inexistentes)
assets/js/ModalManager.js	129	Clase ES6 para modales
assets/js/NotificationManager.js	11	Clase ES6 para notificaciones
assets/js/OrderManager.js	99	Clase ES6 para pedidos

Parecen ser parte de un refactor abandonado. No afectan el funcionamiento del sistema.

13. API REST — Documentación

13.1 POST /menu/confirmar

Crea un pedido y genera ticket de confirmación.

Request (multipart/form-data):

Campo	Tipo	Requerido	Descripción
mesa_id	int	Sí	ID de la mesa
items	string	Sí	JSON: [{"id": "1", "nombre": "Taco", "precio": 25, "cantidad": 2}]

Response 200:

```
{
  "success": true,
  "message": "Pedido enviado a caja. Ticket generado.",
  "pedido_id": 12,
  "total": 42.00,
  "ticket": "☺☺ TAQUERÍA EL INFORMÁTICO\n===== Mesa: M01\nFecha: 24/05/2026\n..."
}
```

Response 400:

```
{
  "success": false,
  "message": "Error al crear el pedido: La mesa no existe"
}
```

13.2 POST /menu/solicitarAsistencia

Solicita asistencia de mesero para una mesa.

Request (multipart/form-data):

Campo	Tipo	Requerido	Descripción
mesa_id	int	Sí	ID de la mesa

Response:

```
{
  "success": true,
  "message": "Asistencia registrada para mesa 1",
  "timestamp": "2026-05-24 14:30:00"
}
```

13.3 GET /menu/getServerDateTime

Obtiene la fecha y hora actual del servidor.

Response:

```
{
  "success": true,
  "fecha": "24/05/2026",
  "hora": "14:30:00",
  "timestamp": 1777050000
}
```

13.4 GET /cocina/obtenerPedidosActualizados

Obtiene pedidos activos filtrados para cocina (solo tacos y postres).

Response:

```
{
  "success": true,
  "pedidos": [
    {
      "id": 1,
      "mesa_id": 1,
      "estado": "pendiente",
      "total": 25.00,
      "detalles": [
        {
          "id": 1,
          "producto_id": 8,
          "cantidad": 1,
          "precio_unitario": 25.00,
          "subtotal": 25.00,
          "nombre": "Taco al pastor",
          "estado": "pendiente"
        }
      ]
    }
  ]
}
```

13.5 POST /cocina/actualizarDetalle

Cambia el estado de un detalle de pedido.

Request (multipart/form-data):

Campo	Tipo	Requerido	Descripción
detalle_id	int	Sí	ID del detalle
estado	string	Sí	Nuevo estado (pendiente/en_proceso/terminado)

Response:

```
{ "success": true }
```

Response error:

```
{ "success": false, "error": "Datos invalidos." }
```

13.6 POST /order/save

API alternativa para guardar pedidos (vía OrderModel).

Request (JSON):

```
{
  "mesa": 1,
  "items": [
    { "id": 8, "nombre": "Taco al pastor", "precio": 25.00, "cantidad": 2 }
  ]
}
```

Response:

```
{ "status": "success", "pedido_id": 12 }
```

14. Imágenes — Especificación Completa

14.1 Ubicación en el Servidor

XAMPP:

/opt/lampp/htdocs/proyecto/sistema_de_comanda_digital/public/images/platillos/

Docker: /var/www/html/public/images/platillos/

Ambas rutas apuntan al mismo directorio relativo dentro del proyecto. El sistema usa `file_exists()` con una ruta relativa a `views/menu/menu.php` para verificar la existencia de imágenes.

14.2 Formato y Dimensiones Recomendados

Atributo	Especificación	Notas
Formato	JPG, JPEG, PNG, WebP	JPG recomendado para fotos, PNG para gráficos
Dimensiones	400 × 300 px	Relación 4:3 (horizontal/apaisado)
Peso máximo	500 KB	Ideal < 200 KB
Modo de color	RGB	sRGB preferido para web
Orientación	Horizontal	Vertical se verá cortado en el contenedor de 200px de alto
Fondo	Claro/neutro	Que contraste con el producto

14.3 Nomenclatura de Archivos

{timestamp_unix}_{nombre_normalizado}.extension

Ejemplos:

- 1776984695_taco.jpg
- 1776992752_tacouri.jpeg
- 1775218844_agua-de-jamaica.jpg

Reglas:

1. **timestamp_unix**: `time()` de PHP (segundos desde el 01/01/1970). Garantiza unicidad.
2. **nombre_normalizado**: Nombre del producto en **minúsculas**, espacios reemplazados por **guión bajo** (`_`). Sin caracteres especiales ni acentos.
3. **extensión**: jpg, jpeg, png, webp (minúsculas).

Ejemplos de nombres correctos:

- 1777050000_taco_al_pastor.jpg
- 1777050000_inca_kola.png
- 1777050000_arroz_con_leche.webp

14.4 Campo en Base de Datos

- **Tabla**: `productos`
- **Columna**: `imagen` (`varchar(255)`, nullable)
- **Valor almacenado**: Solo el nombre del archivo (ej. `1776984695_taco.jpg`)

- **NO almacenar:** Rutas absolutas (`/var/www/html/...`), URLs completas (`http://...`), o paths relativos (`../public/...`)

14.5 Cómo se Guarda una Imagen (desde el Admin)

Formulario (`views/admin/menu.php`):

```
<input type="file" name="imagen" accept="image/*">
```

Procesamiento (`AdminController::agregarProducto()`):

```
private function agregarProducto() {
    $uploadDir = __DIR__ . '/../public/images/platillos/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true); // Crea directorio si no existe
    }

    $imagen = null;
    if (isset($_FILES['imagen']) && $_FILES['imagen']['error'] === UPLOAD_ERR_OK)
    {
        $filename = time() . '_' . basename($_FILES['imagen']['name']);
        move_uploaded_file($_FILES['imagen']['tmp_name'], $uploadDir . $filename);
        $imagen = $filename;
    }

    // Guarda en BD (incluyendo $imagen)
    $productoModel->crear($data, $imagen);
}
```

⚠️ ADVERTENCIA: No hay validación de tipo de archivo, tamaño, ni MIME. Ver [Seguridad](#).

14.6 Cómo se Muestra una Imagen (en el Menú Público)

Verificación (`views/menu/menu.php`):

```
$imagen = $p['imagen'] ?? '';
$imgPath = '';
$tieneImagen = false;
if ($imagen && file_exists(__DIR__ . '/../public/images/platillos/' . $imagen))
{
    $imgPath = BASE_URL . 'public/images/platillos/' . $imagen;
    $tieneImagen = true;
}
```

Renderizado en HTML:

```
<div class="item-image">
    <?php if ($tieneImagen): ?>
        
    <?php else: ?>
        <div class="item-image-placeholder">
            <span>TA</span> <!-- Primeras 2 letras del nombre en mayúsculas -->
        </div>
    <?php endif; ?>
</div>
```

14.7 Estado Actual de Imágenes

Productos activos con imagen:

ID	Producto	Archivo	¿Existe en disco?
----	----------	---------	-------------------

11	taco de perro	1776984695_taco.jpg	✓
12	Taco Uriel	1776992752_tacouri.jpeg	✓

Productos activos SIN imagen:

ID	Producto	Categoría	Precio
4	Inca Kola	Bebidas	\$8.00
8	Taco al pastor	Tacos	\$25.00
9	Taco al pastor con queso	Tacos	\$35.00

Archivos en disco sin producto asociado (huérfanos):

Archivo	Posible producto	Acción recomendada
1775218844_agua-de-jamaica.jpg	Agua de Jamaica (no existe en BD)	Crear producto o eliminar archivo

Imágenes adicionales disponibles en [assets/img/](#):

Archivo	Posible uso
bistec.jpeg	Taco de suadero / bistec
jamaica.avif	Agua de Jamaica
jamaica.png	Agua de Jamaica
pastor.jpeg	Taco al pastor
pastor.webp	Taco al pastor
refrescos.jpeg	Bebidas en general
suadero.jpeg	Taco de suadero

14.8 Procedimiento Recomendado para Agregar Imágenes

Paso 1: Preparar la imagen

```
# Ejemplo con ImageMagick (convert)
convert origen.jpg -resize 400x300^ -gravity center -extent 400x300 producto.jpg

# Comprimir con jpegoptim (Linux)
jpegoptim --max=85 --strip-all producto.jpg

# O usando herramientas online
# TinyPNG (tinypng.com) - compresión PNG/JPG
# Squoosh (squoosh.app) - conversión y compresión
```

Paso 2: Nombrar el archivo

```
# Obtener timestamp actual
php -r "echo time();"
# Resultado: 1777050000

# Nombre final
1777050000_taco_al_pastor.jpg
```


Paso 3: Opción A — Subir desde el Admin

1. Ir a: **Admin** → **Menú**
2. Llenar formulario del producto (o crear nuevo)
3. En el campo "Imagen", seleccionar el archivo preparado
4. Guardar

Paso 4: Opción B — Subir manualmente

```
# Copiar archivo al directorio de imágenes
cp taco_al_pastor.jpg
/opt/lampp/htdocs/proyecto/sistema_de_comanda_digital/public/images/platillos/1777050000_taco_al_pastor.jpg

# Actualizar la base de datos
mysql -u root -e "
USE sistema_comanda_digital_v1;
UPDATE productos SET imagen = '1777050000_taco_al_pastor.jpg' WHERE id = 9;
"
```

Paso 5: Verificar

1. Abrir: http://localhost/proyecto/sistema_de_comanda_digital/menu?mesa=1
2. Buscar el producto y confirmar que la imagen se carga correctamente
3. Verificar que el placeholder (iniciales) NO aparezca para productos con imagen

14.9 Solución de Problemas con Imágenes

Problema	Causa Probable	Solución
No se ve la imagen (alt vacío)	Archivo no existe en disco	Verificar <code>public/images/platillos/</code>
No se ve la imagen (alt vacío)	<code>productos.imagen</code> es NULL	<code>UPDATE productos SET imagen = 'archivo.jpg' WHERE id = N</code>
No se ve la imagen (alt vacío)	Nombre del archivo mal escrito	Verificar que coincida exactamente
Imagen se ve distorsionada	Dimensiones incorrectas	Usar 400×300 px (relación 4:3)
Imagen no se carga (error 404 en consola)	<code>BASE_URL</code> incorrecto en Docker	Verificar <code>config/config.php</code>
Imagen aparece como rota	Permisos de archivo incorrectos	<code>chmod 644 public/images/platillos/*</code>
No se puede subir imagen en admin	Permisos de directorio	<code>chmod 755 public/images/platillos/</code>
No se puede subir imagen en admin	<code>upload_max_filesize</code> en PHP	Aumentar en <code>php.ini</code> : <code>upload_max_filesize = 2M</code>
No se puede subir	<code>post_max_size</code> en PHP	Aumentar en <code>php.ini</code> : <code>post_max_size =</code>

imagen en admin		8M
Placeholder con iniciales para producto con imagen	La imagen se subió pero no se vinculó en BD	Ejecutar UPDATE con el nombre del archivo

14.10 Lista de Verificación para Imágenes

- [] La imagen existe en `public/images/platillos/`
- [] La columna `productos.imagen` contiene el nombre exacto del archivo
- [] El nombre del archivo sigue el formato `{timestamp}_{nombre}.ext`
- [] La imagen tiene dimensiones 400×300 px
- [] La imagen pesa menos de 500 KB
- [] El producto tiene `activo = 1` en la BD
- [] Se verifica la carga en el menú público (`/menu?mesa=1`)

15. Pruebas (PHPUnit)

15.1 Configuración

Archivo: `phpunit.xml`

```
<phpunit bootstrap="tests/bootstrap.php">
  <testsuites>
    <testsuite name="Unit">
      <directory>tests/Unit</directory>
    </testsuite>
  </testsuites>
  <coverage>
    <include>
      <directory>models</directory>
    </include>
    <exclude>
      <directory>models/observers</directory>
    </exclude>
  </coverage>
  <php>
    <env name="APP_ENV" value="testing"/>
    <env name="DB_HOST" value="db"/>
    <env name="DB_NAME" value="comanda_db"/>
    <env name="DB_USER" value="root"/>
    <env name="DB_PASS" value="password"/>
  </php>
</phpunit>
```

15.2 Suites de Prueba

ProductoTest (6 tests)

```
class ProductoTest extends TestCase {
    public function testPrecioConIVA() {
        $precio = 100;
```

```

        $iva = 0.16;
        $this->assertEquals(116, $precio * (1 + $iva));
    }

    public function testValidarNombreProducto() {
        $nombre = 'Taco al pastor';
        $this->assertNotEmpty($nombre);
        $this->assertIsString($nombre);
    }

    public function testValidarPrecioPositivo() {
        $precio = 50;
        $this->assertGreaterThan(0, $precio);
    }

    public function testStockNoNegativo() {
        $stock = 10;
        $this->assertGreaterThanOrEqual(0, $stock);
    }

    public function testCalcularDescuento() {
        $precio = 100;
        $descuento = 10;
        $this->assertEquals(90, $precio - ($precio * $descuento / 100));
    }

    public function testValidarDescripcion() {
        $descripcion = "Taco al pastor sencillo sin verdura ni queso";
        $this->assertLessThanOrEqual(500, strlen($descripcion));
    }
}

```

PedidoTest (6 tests)

```

class PedidoTest extends TestCase {
    public function testCalcularTotalPedido() {
        $items = [
            ['precio' => 50, 'cantidad' => 2],
            ['precio' => 30, 'cantidad' => 1],
            ['precio' => 20, 'cantidad' => 3],
        ];
        $total = array_reduce($items, fn($sum, $item) => $sum + $item['precio'] *
$item['cantidad'], 0);
        $this->assertEquals(190, $total);
    }

    public function testCalcularIVA() {
        $total = 190;
        $this->assertEquals(30.40, $total * 0.16);
    }

    public function testCalcularCambio() {
        $montoPagado = 200;
        $total = 116;
        $cambio = $montoPagado - $total;
        $this->assertGreaterThan(0, $cambio);
        $this->assertEquals(84, $cambio);
    }

    public function testNoCambioNegativo() {

```

```

        $montoPagado = 100;
        $total = 150;
        $cambio = $montoPagado - $total;
        $this->assertLessThan(0, $cambio);
    }
}

```

MesaTest (7 tests)

```

class MesaTest extends TestCase {
    public function testValidarNumeroMesa() {
        $numero = 5;
        $this->assertGreaterThan(0, $numero);
        $this->assertIsInt($numero);
    }

    public function testPorcentajeOcupacion() {
        $ocupadas = 4;
        $total = 10;
        $porcentaje = ($ocupadas / $total) * 100;
        $this->assertEquals(40, $porcentaje);
    }
}

```

InventarioTest (5 tests)

```

class InventarioTest extends TestCase {
    public function testStockSuficiente() {
        $stock = 50;
        $requerido = 10;
        $this->assertGreaterThanOrEqual($requerido, $stock);
    }

    public function testAlertaStockBajo() {
        $stock = 5;
        $minimo = 10;
        $this->assertTrue($stock < $minimo);
    }
}

```

LoggerTest (4 tests)

```

class LoggerTest extends TestCase {
    public function testLogEntryFormat() {
        $entry = json_encode([
            'timestamp' => '2026-05-24T14:30:00-05:00',
            'level' => 'INFO',
            'message' => 'test'
        ]);
        $decoded = json_decode($entry, true);
        $this->assertArrayHasKey('timestamp', $decoded);
        $this->assertArrayHasKey('level', $decoded);
        $this->assertArrayHasKey('message', $decoded);
    }
}

```

15.3 Cómo Ejecutar

```

# Con Composer
composer run-script test

```

```
# Con PHPUnit directamente
./vendor/bin/phpunit

# Con script personalizado
bash scripts/run-tests.sh

# Con cobertura
./vendor/bin/phpunit --coverage-html coverage/

# Solo un test específico
./vendor/bin/phpunit --filter testCalcularTotalPedido
```

16. Seguridad

16.1 Vulnerabilidades Identificadas

#	Vulnerabilidad	Archivo	Línea(s)	Severidad	Impacto
1	Backdoor contraseña "123456"	<code>models/User.php</code>	22	 Alta	Cualquier usuario puede iniciar sesión con "123456"
2	Sin validación de archivos subidos	<code>AdminController.php</code>	~200	 Alta	Se puede subir cualquier tipo de archivo (.php, .exe)
3	Sin CSRF tokens en formularios	Todos los forms	General	 Alta	Ataques de Cross-Site Request Forgery
4	Acceso sin autenticación a mesero/cocina/caja	Controladores	-	 Alta	Cualquier persona puede acceder a URLs internas
5	<code>.htaccess</code> typo <code>!-dy</code>	<code>.htaccess</code>	5	 Media	Verificación de directorios no funciona
6	Logger usa <code>\$_SESSION['user_id']</code> incorrecto	<code>helpers/Logger.php</code>	7	 Media	<code>user_id</code> siempre null en logs
7	XSS potencial en varios <code>echo</code>	Varios	Varias	 Media	Posible inyección de scripts en salidas sin escapar
8	<code>usuario_id</code> hardcodeado	<code>MenuController.php</code>	46	 Media	Todos los pedidos se asocian a

					usuario_id=2
9	usuario_id hardcodeado	CajaController.php	~40	Media	Todos los pagos se asocian a usuario_id=1
10	OrderModel con credenciales hardcodeadas	models/OrderModel.php	10	Media	Credenciales root en texto plano
11	Sin transacción en operaciones admin	AdminController.php	Múltiples	Baja	Operaciones parciales si algo falla
12	Filtro cocina por nombre (case-insensitive)	CocinaController.php	~70	Baja	Productos mal nombrados pueden no aparecer

16.2 Recomendaciones de Seguridad Prioritarias

Críticas (implementar inmediatamente)

1. Eliminar backdoor de contraseña

En `models/User.php`, línea 22:

```
// CAMBIAR:
if ($usuario && (password_verify($password, $usuario['password_hash']) ||
$password === '123456')) {

// POR:
if ($usuario && password_verify($password, $usuario['password_hash'])) {
```

2. Agregar validación de archivos subidos

En `AdminController.php`:

```
private function validarImagen($file) {
    $allowedTypes = ['image/jpeg', 'image/png', 'image/webp'];
    $maxSize = 2 * 1024 * 1024; // 2MB

    if ($file['error'] !== UPLOAD_ERR_OK) return false;
    if ($file['size'] > $maxSize) return false;

    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mime = finfo_file($finfo, $file['tmp_name']);
    finfo_close($finfo);

    if (!in_array($mime, $allowedTypes)) return false;

    $extension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
    if (!in_array($extension, ['jpg', 'jpeg', 'png', 'webp'])) return false;

    return true;
}
```

3. Agregar verificación de autenticación

En `MeseroController`, `CocinaController`, `CajaController`:

```
public function __construct() {
    if (!isset($_SESSION['usuario_id'])) {
        header('Location: ' . BASE_URL . 'index.php?action=login');
        exit;
    }
}
```

4. Agregar CSRF tokens

```
// Generar token en el formulario
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
echo '<input type="hidden" name="csrf_token" value="' . $_SESSION['csrf_token'] .
'>';

// Verificar al procesar
if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !==
$_SESSION['csrf_token']) {
    die('CSRF token inválido');
}
```

17. Mantenimiento

17.1 Logs del Sistema

Archivo	Contenido	Rotación
<code>logs/app.log</code>	Todos los niveles (INFO, WARNING, ERROR, DEBUG)	Manual
<code>logs/error.log</code>	Solo ERROR y CRITICAL	Manual
<code>logs/sistema.log</code>	Eventos del Observer (stock bajo, cambios inventario, nuevo pedido)	Manual

Formato de log (JSON):

```
{
  "timestamp": "2026-05-24T14:30:00-05:00",
  "level": "INFO",
  "message": "Usuario autenticado",
  "context": {"user": "admin"},
  "ip": "192.168.1.1",
  "user_id": null
}
```

 **BUG:** `user_id` siempre será `null` porque el Logger referencia `$_SESSION['user_id']` pero la sesión guarda `$_SESSION['usuario_id']`.

17.2 Backup de Base de Datos

Script incluido (`scripts/backup.sh`):

```
#!/bin/bash
docker exec sistema_de_comanda_digital-db-1 mysqldump -uroot -proot \
    sistema_comanda_digital_v1 | gzip > "backup_$(date +%F_%H-%M-%S).sql.gz"
```

```
# Retención: elimina backups con más de 7 días
find . -name "backup_*.sql.gz" -mtime +7 -delete
```

Uso:

```
bash scripts/backup.sh
```

Restaurar:

```
gunzip < backup_2026-05-24_14-30-00.sql.gz | docker exec -i
sistema_de_comanda_digital-db-1 mysql -uroot -proot sistema_comanda_digital_v1
```

17.3 Alertas del Sistema

La tabla `alertas_sistema` almacena notificaciones automáticas generadas por el Observer cuando se detecta stock bajo.

Consultar alertas no leídas:

```
SELECT * FROM alertas_sistema WHERE leida = 0 ORDER BY fecha_creacion DESC;
```

Marcar alertas como leídas:

```
UPDATE alertas_sistema SET leida = 1 WHERE id IN (1, 2, 3);
```

17.4 Tareas Periódicas Recomendadas

Frecuencia	Tarea	Comando
Diaria	Backup BD	<code>bash scripts/backup.sh</code>
Semanal	Revisar logs	<code>tail -100 logs/error.log</code>
Semanal	Verificar stock bajo	<code>SELECT * FROM alertas_sistema WHERE leida = 0</code>
Mensual	Limpiar logs antiguos	<code>find logs/ -name "*.log" -mtime +30 -delete</code>
Mensual	Revisar ventas canceladas	<code>SELECT * FROM ventas WHERE estado = 'cancelado'</code>

18. Troubleshooting

18.1 Problemas de Instalación

Problema	Causa	Solución
Página en blanco al acceder	PHP no ejecuta	Verificar que Apache tenga PHP habilitado
Error 404 en todas las rutas	<code>mod_rewrite</code> deshabilitado	<code>a2enmod rewrite && systemctl restart apache2</code>
"No hay mesas activas"	BD no inicializada	Importar <code>init-db/sistema_comanda_digital_v1.sql</code>
Error de conexión BD	Credenciales incorrectas	Verificar <code>config/config.php</code>
Error de conexión BD en Docker	MySQL no listo	Esperar 30s, verificar <code>docker-compose logs db</code>

Docker: "port already allocated"	Puerto 8081/3307 en uso	Cambiar puertos en <code>docker-compose.yml</code>
Docker: permisos denegados	SELinux	Agregar <code>:z</code> a volúmenes en <code>docker-compose.yml</code>

18.2 Problemas de Frontend

Problema	Causa	Solución
Estilos no cargan (CSS)	<code>ASSETS_URL</code> incorrecto	Verificar <code>config/config.php</code> y <code>BASE_URL</code>
JS no funciona	Archivo JS no encontrado	Verificar referencia en vista
Carrito no se abre	<code>floatingCart</code> es null	Verificar que el elemento <code>#floatingCart</code> existe en HTML
"Agregar" no funciona	Botones no tienen <code>data-id</code>	Verificar que el producto tiene ID en BD
Notificaciones no aparecen	CSS <code>.notification</code> no definido	Verificar <code>estilos.css</code> línea 644+
Modal de carrito no se ve	Z-index conflict	Verificar <code>z-index: 1001</code> en <code>.cart-modal</code>

18.3 Problemas de Pedidos

Problema	Causa	Solución
Pedido no se crea	Error de BD	Verificar logs de PHP/Apache
Pedido no aparece en cocina	Producto no contiene "taco" o "postre"	Cocina solo filtra tacos y postres
Pedido no aparece en mesero	Pedido está en estado "entregado" o "cancelado"	Mesero solo muestra pedidos activos
Error "La mesa no existe"	Mesa inactiva o ID incorrecto	Verificar <code>mesas.activa = 1</code>
Error 400 al confirmar	JSON de items mal formado	Verificar <code>JSON.stringify(cart)</code> en JS

18.4 Problemas de Login

Problema	Causa	Solución
No redirige después de login	<code>session_regenerate_id()</code> conflicto	Limpiar cookies del navegador
Error "Usuario o contraseña incorrectos"	Hash incorrecto	Usar contraseña "password" o "123456" (backdoor)
Sesión expira constantemente	<code>session.gc_maxlifetime</code> bajo	Aumentar en <code>php.ini</code> : <code>session.gc_maxlifetime = 86400</code>

18.5 Problemas de Docker

Problema	Comando de diagnóstico
Ver logs del contenedor web	<code>docker-compose logs web</code>
Ver logs de MySQL	<code>docker-compose logs db</code>
Acceder a MySQL	<code>docker exec -it sistema_de_comanda_digital-db-1 mysql -uroot -proot</code>
Ver archivos en el contenedor	<code>docker exec -it sistema_de_comanda_digital-web-1 ls -la /var/www/html/</code>
Verificar conexión web→db	<code>docker exec sistema_de_comanda_digital-web-1 ping db</code>
Reconstruir contenedores	<code>docker-compose build --no-cache && docker-compose up -d</code>

19. Referencia Rápida

19.1 Comandos Útiles

```
# — DOCKER —————
docker-compose up -d           # Iniciar servicios
docker-compose down           # Detener servicios
docker-compose down -v        # Detener y eliminar volúmenes (borra BD)
docker-compose logs -f        # Ver logs en tiempo real
docker-compose build --no-cache # Reconstruir imágenes

# — MYSQL —————
# Importar BD (XAMPP)
mysql -u root < init-db/sistema_comanda_digital_v1.sql

# Acceder a MySQL (XAMPP)
mysql -u root

# Acceder a MySQL (Docker)
docker exec -it sistema_de_comanda_digital-db-1 mysql -uroot -proot

# Backup (Docker)
docker exec sistema_de_comanda_digital-db-1 mysqldump -uroot -proot
sistema_comanda_digital_v1 > backup.sql

# — PHP —————
# Verificar sintaxis
php -l controllers/MenuController.php

# — TESTS —————
./vendor/bin/phpunit           # Ejecutar todos los tests
./vendor/bin/phpunit --filter testCalcularTotalPedido # Test específico

# — LOGS —————
tail -f logs/app.log           # Ver logs de la app en tiempo real
tail -f logs/error.log         # Ver errores en tiempo real
```

19.2 URLs de Acceso

URL	Descripción	Autenticación
<code>http://localhost:8081/mesa</code> (Docker)	Selección de mesa	No
<code>http://localhost/proyecto/.../mesa</code> (XAMPP)	Selección de mesa	No
Cualquiera + <code>/login</code>	Inicio de sesión	No
Cualquiera + <code>/admin</code>	Panel de administración	admin
Cualquiera + <code>/mesero</code>	Panel de mesero	No (sin auth)
Cualquiera + <code>/cocina</code>	Panel de cocina	No (sin auth)
Cualquiera + <code>/caja</code>	Panel de caja	No (sin auth)

19.3 Credenciales de Prueba

Usuario	Contraseña	Rol
admin	password	Administrador
mesero1	password	Mesero
cocina1	password	Cocina
caja	password	Caja

20. Bugs Conocidos

#	Bug	Archivo	Línea	Descripción	Impacto
1	<code>!-dy</code> en lugar de <code>!-d</code>	<code>.htaccess</code>	5	Typo en condición de directorio	Bajo — no afecta funcionalidad
2	Backdoor "123456"	<code>models/User.php</code>	22	Cualquier contraseña "123456" funciona	Alto — cualquier a puede acceder
3	<code>\$_SESSION['user_id']</code> incorrecto	<code>helpers/Logger.php</code>	7	Debería ser <code>\$_SESSION['usuario_id']</code>	Bajo — user_id siempre null
4	<code>\$this->orderModel->pdo</code> privado	<code>controllers/OrderController.php</code>	~50	Acceso a propiedad privada	Medio — puede fallar en PHP estricto
5	<code>getPendingOrder</code>	<code>models/OrderModel.php</code>	~60	Método declarado pero	Alto —

	<code>s()</code> sin implementar			vacío	error fatal si se invoca
6	<code>\$usuario_id = 2</code> hardcodeado	<code>controllers/MenuController.php</code>	46	Todos los pedidos usan <code>mesero1</code>	Medio — mezcla datos de usuarios
7	<code>\$usuario_id = 1</code> hardcodeado	<code>controllers/CajaController.php</code>	~40	Todos los pagos usan <code>admin</code>	Medio — mezcla datos de usuarios
8	Sin verificación de auth en staff	<code>MeseroController</code> , <code>CocinaController</code> , <code>CajaController</code>	-	Cualquiera puede acceder	Alto — sin protección
9	Sin validación de tipo de archivo	<code>AdminController.php</code>	~200	Se puede subir cualquier archivo	Alto — riesgo de seguridad
10	<code>BASE_URL</code> hardcodeada en <code>cocina.js</code>	<code>views/cocina/cocina.php</code>	~50	<code>http://localhost/comanda1/</code>	Alto — no funciona en Docker
11	Zona horaria BD hardcodeada	<code>config/database.php</code>	20	<code>SET time_zone = '-05:00'</code>	Bajo — incorrecto para otras zonas
12	<code>OrderModel</code> credenciales hardcodeadas	<code>models/OrderModel.php</code>	10	<code>new PDO('mysql:host=localhost', 'root', '')</code>	Medio — no funciona en Docker